

HelixLLM

HelixLLM

Ein Binärprogramm, sechs Modi - OpenAI- und Anthropic-kompatible Inferenz von Ihrem Laptop bis zum Multi-Host-Cluster.

Zusammenfassung

HelixLLM ist ein unternehmensfähiges, verteiltes LLM-System in Go: ein einziges Binärprogramm mit einem Modussystem, das von der Einzelhost-Entwicklung bis zur Multi-Host-Produktion skaliert. Es bietet vollständig OpenAI- und Anthropic-kompatible APIs über HTTP/3, lokale llama.cpp-Inferenz, eine bewertete Multi-Provider-Fallback-Kette, eine RAG-Pipeline und ein ReAct-Agentensystem.

Kurzbeschreibung

HelixLLM ist ein einzelnes Binärprogramm, ein Go-basiertes verteiltes LLM-System. Es stellt OpenAI- und Anthropic-kompatible APIs über HTTP/3 bereit, führt lokale llama.cpp-Inferenz aus, erkennt und bewertet kostenlose Cloud-Anbieter automatisch für eine Failover-Kette und integriert eine RAG-Wissenspipeline sowie einen werkzeugnutzenden ReAct-Agenten – einsetzbar in sechs Modi.

Ausführliche Beschreibung

HelixLLM ist ein unternehmensfähiges, verteiltes LLM-System, entwickelt in Go mit Gin. Sein zentraler Vorteil besteht darin, dass ein einziges Artefakt für jede Skalierungsstufe genutzt wird. Es kompiliert zu einem einzelnen Binärprogramm, dessen Modussystem zur Bereitstellungszeit entscheidet, was dieses Programm *ist*: Führen Sie es als `full` für eine All-in-One-Instanz auf einem Laptop aus oder verteilen Sie die Aufgaben auf die Modi `gateway`, `brain`, `knowledge`, `agents` und `control`, die über

mehrere Hosts verteilt sind – derselbe Code, nur neu angeordnet statt umgeschrieben, von der Entwicklermaschine bis zum Produktionscluster.

Es beherrscht zwei Dialekte fließend: vollständig OpenAI- und Anthropic-kompatible APIs, sodass bestehende SDK-Clients aus beiden Ökosystemen ohne Anpassungen funktionieren. Alle Dienste werden über HTTP/3 (QUIC) mit automatischem HTTP/2-Fallback und TLS 1.3 bereitgestellt. Die lokale Inferenz läuft über llama.cpp mit CUDA-, Metal- und ROCm-Unterstützung, sodass dieselbe Build auf Nvidia-, Apple- und AMD-Hardware beschleunigt wird. Besonders hervorzuheben ist die Multi-Provider-Fallback-Kette, die die notorische Unzuverlässigkeit kostenloser Cloud-Inferenz in eine verwaltete, selbstheilende Ressource verwandelt: HelixLLM erkennt automatisch kostenlose Modelle von über sieben Cloud-Anbietern (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), bewertet sie alle fünf Minuten mittels LLMsVerifier und leitet Anfragen durch die bewertete Kette mit automatischem Failover bei 429/5xx-Fehlern – immer mit lokalem llama.cpp als garantiertem letzten Ausweg, sodass eine Anfrage niemals einfach aufgrund fehlender verfügbarer Anbieter scheitert.

Über die reine Inferenz hinaus ist HelixLLM eine vollständige Anwendungsplattform: Eine RAG-Wissenspipeline (Aufnahme, Zerlegung, Einbettung, vector-Suche) und ein ReAct-Agentensystem mit Werkzeugaufrufen, Konversationssitzungen und RAG-Integration sind im selben Binärprogramm enthalten. Das Modusystem zahlt sich auch auf Netzwerkebene aus – im full-Modus kommunizieren alle Schichten über direkte In-Prozess-Go-Aufrufe ohne Netzwerk-Overhead, während dasselbe Binärprogramm, auf mehrere Hosts verteilt, über gRPC, SSE und Kafka koordiniert. Abgerundet wird das Ganze durch Brotli/gzip-Inhaltsaushandlung, SSE-Streaming, das die OpenAI- und Anthropic-Formate Byte für Byte abbildet, API-Key- und JWT-Authentifizierung mit Ratenbegrenzung, Prometheus-Metriken, OpenTelemetry-Tracing sowie eine umfangreiche Sammlung produktionsreifer Go-Submodule.

Warum wir es entwickelt haben

Teams benötigen Inferenz, die portabel, standardkompatibel und resilient ist – ohne Clients umschreiben oder sich an einen einzigen Anbieter oder eine einzige Maschine binden zu müssen. HelixLLM wurde entwickelt, damit dasselbe Binary lokal für die Entwicklung laufen und auf einen Multi-Host-Produktionscluster skalieren kann, wobei es die OpenAI- und Anthropic-Dialekte spricht, die Clients bereits nutzen.

Warum es ein Game-Changer ist

Es reduziert einen gesamten Inferenz-Stack – Gateway, lokale Inferenz, Cloud-Fallback, RAG und Agenten – auf ein einziges Binary, das per Modus-Schalter gesteuert wird. So wird die Architektur, die Sie einsetzen, zur Laufzeitentscheidung statt zu einem Re-Plattforming-Projekt. Und es verwandelt etwas, das bisher ein Risiko war, in ein Feature: Die Zuverlässigkeit von Cloud-Anbietern wird zu einem erstklassigen, kontinuierlich gemessenen Anliegen, das durch eine bewertete, selbstheilende Fallback-Kette gehandhabt wird. Diese stuft Anbieter alle paar Minuten neu ein und fällt stets auf garantierte lokale Inferenz zurück. Die dadurch freigeschaltete Fähigkeit ist ein einziger Endpunkt, auf den Sie sich wirklich verlassen können – standardkompatibel, portabel vom Laptop zum Cluster und unfähig, auszufallen, nur weil ein vorgelagerter Anbieter ratenlimitiert oder versagt.

Was innovativ ist

- Ein einziges Binary mit einem Sechs-Modus-System, das entweder als All-in-One oder in verteilten Rollen läuft – direkte In-Process-Go-Aufrufe im full-Modus, gRPC/SSE/Kafka bei getrennter Ausführung – sodass sich die Bereitstellungstopologie ändern lässt, ohne Code zu ändern oder eine ungewollte Netzwerkbelastung zu verursachen.
- Eine bewertete, automatisch entdeckende Multi-Anbieter-Fallback-Kette mit über 7 kostenlosen Anbietern, die kontinuierlich nach LLMsVerifier eingestuft werden, mit automatischem 429/5xx-Failover und einem garantierten llama.cpp-Notfallplan – kostenlose Kapazitäten werden zu verlässlichen Kapazitäten.
- Duale OpenAI- und Anthropic-kompatible Schnittstellen, die über HTTP/3 bereitgestellt werden, mit automatischem HTTP/2-Fallback, sodass Clients aus beiden Ökosystemen ohne Anpassungen verbunden werden können.
- Lokale Inferenz, die CUDA, Metal und ROCm aus einer Codebasis unterstützt – derselbe Build läuft beschleunigt auf Nvidia-, Apple- und AMD-Hardware.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Skalierung von einem Host auf viele ohne Neuschreiben.** Die meisten Systeme erzwingen eine harte Grenze zwischen „lokaler Entwicklung“ und „verteilter Produktion“, und der Übergang bedeutet eine Neugestaltung der Architektur. Wir haben diese Grenze mit einem Modus-System in einem

einzigsten Binary aufgehoben: Dieselben Schichten kommunizieren im full-Modus über direkte In-Process-Aufrufe und wechseln in verteilten Modi transparent zu gRPC/SSE/Kafka, sodass das Skalieren eine Konfigurationsänderung statt eines Portierungsprojekts ist.

- **Unzuverlässige, ratenlimitierte kostenlose Cloud-Anbieter.** Kostenlose Inferenz ist schnell – bis sie mit 429-Fehlern abbricht oder mitten in einer Anfrage verschwindet. Wir haben sie zuverlässig gemacht, indem wir verfügbare Modelle automatisch erkennen, sie mit LLMsVerifier bewerten, Ratenlimit-Header proaktiv überwachen, um Anbieter zu umgehen, die kurz vor dem Throttling stehen, und automatisch entlang der bewerteten Kette auf lokale llama.cpp-Inferenz zurückfallen – sodass die Unbeständigkeit des Pools nie beim Aufrufer ankommt.
- **Client-Kompatibilität über zwei Ökosysteme hinweg.** Clients für ein neues Inferenz-Backend umzuschreiben, ist keine Option. Wir haben sowohl die OpenAI- *als auch* die Anthropic-API-Schnittstellen implementiert – bis hin zu ihren unterschiedlichen SSE-Streaming-Formaten –, sodass SDKs aus beiden Lagern auf HelixLLM zeigen und einfach funktionieren.

Technologie-Stack

- **Go + Gin** – ausgewählt, weil ein Single-Binary-Ansatz mit Fokus auf Nebenläufigkeit das gesamte Modusystem erst ermöglicht: ein Build, der sowohl als Laptop-Server als auch als Cluster-Knoten fungieren kann. Er trägt das gesamte System sowie die HTTP-Schicht des Gateways.
- **HTTP/3 (QUIC) + TLS 1.3 mit HTTP/2-Fallback** – gewählt für modernen, latenzarmen und verbindungsresistenten Transport, der als Server-Oberfläche mit automatischer Aushandlung exponiert wird, sodass Clients, die QUIC nicht unterstützen, nahtlos auf HTTP/2 zurückfallen.
- **llama.cpp (CUDA/Metal/ROCm)** – ausgewählt für portable lokale Inferenz, die über einen einzigen Codebase Nvidia-, Apple- und AMD-Backends beschleunigt; dient gleichzeitig als garantierter Notfalleanbieter, der verhindert, dass die Fallback-Kette jemals vollständig versagt.
- **LLMsVerifier** – eingesetzt, um die Frage „Welcher Anbieter ist aktuell der beste?“ in eine Zahl zu übersetzen; bewertet und sortiert die Cloud-Fallback-Kette alle fünf Minuten neu, sodass das Routing auf Echtzeitqualität statt veralteter Annahmen basiert.
- **Cloud-Anbieter (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together)** – genutzt, um kostenlose Kapazitäten verschiedener Upstream-Quellen zu erschließen; werden automatisch erkannt und zu einer

einzigsten Failover-Kette zusammengefasst, sodass kein Anbieter zum Single Point of Failure wird.

- **gRPC + SSE + Kafka** – gewählt als Transportprotokolle für verteilte Bereitstellungen: gRPC für Service-zu-Service-Aufrufe, SSE für Streaming und Kafka für entkoppelte Ereignisflüsse zwischen den Rollen.
- **Vektorspeicher / embeddings** – eingesetzt, um die Wissenspipeline RAG von Anfang bis Ende zu betreiben: Ingestion, Chunking, Embedding und Suche über Dokumente, die die Antworten des Modells fundieren.
- **Prometheus + OpenTelemetry** – gewählt für Metriken und verteilte Tracing-Funktionen, die eine Anfrage über alle eingesetzten Modi hinweg verfolgen.
- **vasic-digital-Go-Submodule** – genutzt, um erprobte Infrastrukturprimitive aus der Produktion wiederzuverwenden, statt sie neu zu entwickeln, und so die Konsistenz des Systemfundaments mit dem breiteren Stack zu wahren.

Status & Transparenzhinweise

- **Status: Beta.** Funktionsfähiges, aktiv weiterentwickeltes verteiltes Inferenzsystem.
- **Lizenz: Ungeklärt.** Das Repository enthält keine Lizenzangabe in seinen Metadaten (`licenseInfo null`) – dieser Zustand ist UNBESTÄTIGT und muss vor einer Lizenzaussage geklärt werden.
- Das kanonische Repository verweist aktuell auf github.com/HelixDevelopment/llm; der Pfad `HelixLLM` leitet dorthin um. Die im README genannten Angaben zu Testabdeckung und Submodul-Anzahl stammen aus Eigenangaben.

Prioritätsstufe: Helix-primär.